I want to run bare metal os on raspberry pi 5 and for that i want ot enable uart on pi 5 using rp1 peripherals I wan you to write code in c or rust to start uart and connect it using minicom find relevant resources

Here is a minimal bare-metal recipe to bring up an RP1 UART on Raspberry Pi 5 (GPIO header pins 8/10), plus C and Rust examples that initialize the DesignWare/16550-style UART at 115200 8N1 and instructions for connecting with minicom on a host PC.[1] [2] [3]

What runs where

- Raspberry Pi 5 exposes its GPIO, I2C, SPI, PWM, and the 5× UARTs through the RP1 southbridge over PCIe; the AP (BCM2712) must access RP1 peripherals via a PCIe BAR window that maps RP1's internal APB/AXI address space.[4] [1]
- In the RP1 datasheet, UART instances are at uart0..uart5 with processor-view bases 0x40030000..0x40044000; when seen from the host (AP) over PCIe BAR1, the offsets are the same minus 0x40000000, so uart0 is at BAR1 + 0x00030000, and IO_BANK0 (GPIO mux) is at BAR1 + 0x000D0000, which matches the Linux device tree nodes like serial@30000 and gpio@d0000 on Pi 5.[3] [1]
- RP1 UARTs use a DesignWare/16550-style register layout; a typical baud reference is clk_uart ≈ 48 MHz, so a simple integer divisor for 115200 baud is $ div = \lfloor \frac{48,MHz}{16 \times 115200} \rfloor = 26 $, which is close enough for terminal work; adjust if a different clk_uart is configured on the board.[1]

Minimal bring-up steps

- Ensure the PCIe root complex is enabled and RP1 endpoint is enumerated; read RP1's BAR1 and map it to obtain a CPU virtual/physical base for MMIO, as Linux and reference drivers do by mapping BAR1 to reach the APB windows.[4]
- Compute peripheral bases relative to BAR1: UART0 = BAR1 + 0x30000, UART1 = BAR1 + 0x34000, ..., IO_BANK0 = BAR1 + 0xD0000, and optionally PADS_BANK0 = BAR1 + 0xF0000 for pulls/drive.[3] [1]
- Set GPIO alt functions for pins 14 (TX) and 15 (RX) to the UART function in IO_BANK0's GPIO14_CTRL and GPIO15_CTRL FUNCSEL fields; the datasheet table lists the available functions per pin and shows UART0_TX/RX on GPIO14/15 so select the corresponding FUNCSEL values for those pins.[1]

- Initialize the DW_apb/16550 UART: set DLAB, program DLL/DLH for the chosen baud, set LCR to 8N1, enable FIFOs via FCR, then poll LSR for TX holding-empty on transmit and data-ready on receive.[1]

C example (UART0 on GPIO 14/15, 115200 8N1)

```
// Bare-metal RP1 UART0 init on Raspberry Pi 5 (GPIO14/15), 115200 8N1.
// Assumptions:
//  - PCIe and RP1 are up; BAR1 is mapped at RP1_BAR1 (discover via PCIe ECAM).
//  - clk_uart ≈ 48 MHz on RP1; integer divisor 26 yields ~115200 baud.
//  - FUNCSEL values for GPIO14/15 are set per RP1 Table 4 to route UART0 TX/RX.
// See: RP1 datasheet (peripheral map, GPIO function select, UART registers).

#include <stdint.h>

#define RP1_BAR1_BASE   0x40000000UL  // PLACEHOLDER: replace with actual mapped BAR1 bas
#define RP1_UART0_OFF   0x00030000UL  // uart0 offset within BAR1
#define RP1_IOBANK0_OFF 0x000D0000UL  // io_bank0 offset within BAR1
#define RP1_PADS0_OFF   0x000F0000UL  // pads_bank0 offset within BAR1 (optional)

#define UART0_BASE   (RP1_BAR1_BASE + RP1_UART0_OFF)
#define IOBANK0_BASE (RP1_BAR1_BASE + RP1_IOBANK0_OFF)
#define PADS0_BASE   (RP1_BAR1_BASE + RP1_PADS0_OFF)

// 16550/DW_apb UART registers (32-bit spaced)
#define UART_RBR   (*(volatile uint32_t *)(UART0_BASE + 0x00)) // RX buffer (read)
#define UART_THR   (*(volatile uint32_t *)(UART0_BASE + 0x00)) // TX holding (write)
#define UART_DLL   (*(volatile uint32_t *)(UART0_BASE + 0x00)) // Divisor LSB when DLAB=1
#define UART_IER   (*(volatile uint32_t *)(UART0_BASE + 0x04))
#define UART_DLH   (*(volatile uint32_t *)(UART0_BASE + 0x04)) // Divisor MSB when DLAB=1
#define UART_IIR   (*(volatile uint32_t *)(UART0_BASE + 0x08))
#define UART_FCR   (*(volatile uint32_t *)(UART0_BASE + 0x08))
#define UART_LCR   (*(volatile uint32_t *)(UART0_BASE + 0x0C))
#define UART_MCR   (*(volatile uint32_t *)(UART0_BASE + 0x10))
#define UART_LSR   (*(volatile uint32_t *)(UART0_BASE + 0x14))
#define UART_USR   (*(volatile uint32_t *)(UART0_BASE + 0x7C)) // optional (DW_apb)

static inline void udelay(volatile uint32_t n) { while (n--) __asm__ volatile("nop"); }

// IO_BANK0: each GPIO has STATUS at base+0x0 + 8*n and CTRL at base+0x4 + 8*n
#define GPIO_CTRL(n) (*(volatile uint32_t *)(IOBANK0_BASE + 0x4 + 8*(n)))

// Replace with the correct FUNCSEL codes for UART0 TX/RX on GPIO14/15 from RP1 Table 4.
#define FUNCSEL_UART0_TX_GPIO14  4u   // placeholder based on table position
#define FUNCSEL_UART0_RX_GPIO15  4u   // placeholder based on table position
#define GPIO_CTRL_FUNCSEL_MASK   0x1Fu // assume 5-bit FUNCSEL like RP-class parts

static void rp1_uart0_gpio14_15_mux(void) {
    // GPIO14 -> UART0_TX
    uint32_t c14 = GPIO_CTRL(14);
    c14 = (c14 & ~GPIO_CTRL_FUNCSEL_MASK) | FUNCSEL_UART0_TX_GPIO14;
    GPIO_CTRL(14) = c14;

    // GPIO15 -> UART0_RX
    uint32_t c15 = GPIO_CTRL(15);
```

```
    c15 = (c15 & ~GPIO_CTRL_FUNCSEL_MASK) | FUNCSEL_UART0_RX_GPIO15;
    GPIO_CTRL(15) = c15;

    // Optional: configure pulls/drive in PADS_BANK0 if needed (e.g., RX pull-up)
    (void)PADS0_BASE;
}

static void uart0_init_115200_8n1(void) {
    // Disable interrupts
    UART_IER = 0x00;

    // Enable FIFOs and clear RX/TX
    UART_FCR = 0x07;

    // Set DLAB to access divisor, 8-bit
    UART_LCR = 0x80 | 0x03; // DLAB=1, 8N1

    // clk_uart ~ 48MHz => divisor ≈ 26 for 115200
    UART_DLL = 26;  // LSB
    UART_DLH = 0;   // MSB

    // Clear DLAB, keep 8N1
    UART_LCR = 0x03;

    // No modem control
    UART_MCR = 0x00;
}

static void uart0_putc(char c) {
    // Wait for THR empty
    while ((UART_LSR & (1u << 5)) == 0) { }
    UART_THR = (uint32_t)c;
}

static void uart0_puts(const char *s) {
    while (*s) {
        if (*s == '\n') uart0_putc('\r');
        uart0_putc(*s++);
    }
}

void uart_demo(void) {
    rp1_uart0_gpio14_15_mux();
    uart0_init_115200_8n1();
    uart0_puts("RP1 UART0 hello at 115200 8N1\r\n");
    for (;;) {
        // Echo loop
        if (UART_LSR & 0x01) {
            char c = (char)UART_RBR;
            uart0_putc(c);
        }
    }
}
```

This targets UART0 on pins GPIO14/15 via RP1; if using a different UART instance (e.g., UART1..UART5), adjust the base offset to 0x34000..0x44000 and select the matching FUNCSEL values shown for those GPIOs in the RP1 datasheet's function-select table.[3] [1]

Rust example (unsafe, same registers)

```
// Bare-metal RP1 UART0 init in Rust (AArch64), unsafe MMIO.
// See notes in the C example for assumptions.

const RP1_BAR1_BASE: usize = 0x4000_0000; // PLACEHOLDER: replace with actual mapped BAR1
const UART0_OFF: usize = 0x0003_0000;
const IOBANK0_OFF: usize = 0x000D_0000;

const UART_BASE: usize = RP1_BAR1_BASE + UART0_OFF;
const IOBANK0_BASE: usize = RP1_BAR1_BASE + IOBANK0_OFF;

#[inline(always)]
unsafe fn mmio32(addr: usize) -> *mut u32 { addr as *mut u32 }

#[inline(always)]
unsafe fn reg_read(addr: usize) -> u32 { core::ptr::read_volatile(mmio32(addr)) }

#[inline(always)]
unsafe fn reg_write(addr: usize, val: u32) { core::ptr::write_volatile(mmio32(addr), val)

fn gpio_ctrl_offset(n: usize) -> usize { IOBANK0_BASE + 0x4 + 8*n }

const FUNCSEL_MASK: u32 = 0x1F; // adjust if different
const FUNC_UART0_TX_GPIO14: u32 = 4; // placeholder per datasheet table
const FUNC_UART0_RX_GPIO15: u32 = 4; // placeholder per datasheet table

fn gpio_uart_mux() {
    unsafe {
        // GPIO14 -> UART0_TX
        let o14 = gpio_ctrl_offset(14);
        let mut c14 = reg_read(o14);
        c14 = (c14 & !FUNCSEL_MASK) | FUNC_UART0_TX_GPIO14;
        reg_write(o14, c14);

        // GPIO15 -> UART0_RX
        let o15 = gpio_ctrl_offset(15);
        let mut c15 = reg_read(o15);
        c15 = (c15 & !FUNCSEL_MASK) | FUNC_UART0_RX_GPIO15;
        reg_write(o15, c15);
    }
}

fn uart_reg(off: usize) -> usize { UART_BASE + off }
const RBR_THR_DLL: usize = 0x00;
const IER_DLH: usize     = 0x04;
const FCR_IIR: usize     = 0x08;
const LCR: usize         = 0x0C;
const MCR: usize         = 0x10;
const LSR: usize         = 0x14;
```

```
fn uart_init_115200() {
    unsafe {
        // IER = 0
        reg_write(uart_reg(IER_DLH), 0);

        // FCR: enable FIFO, clear RX/TX
        reg_write(uart_reg(FCR_IIR), 0x07);

        // LCR: DLAB=1 | 8N1
        reg_write(uart_reg(LCR), 0x80 | 0x03);

        // Divisor = 26 (48MHz / (16*115200))
        reg_write(uart_reg(RBR_THR_DLL), 26); // DLL
        reg_write(uart_reg(IER_DLH), 0);      // DLH

        // LCR: DLAB=0, 8N1
        reg_write(uart_reg(LCR), 0x03);

        // MCR: none
        reg_write(uart_reg(MCR), 0x00);
    }
}

fn uart_putc(c: u8) {
    unsafe {
        while (reg_read(uart_reg(LSR)) & (1 << 5)) == 0 {}
        reg_write(uart_reg(RBR_THR_DLL), c as u32);
    }
}

pub fn uart_demo() -> ! {
    gpio_uart_mux();
    uart_init_115200();
    for &b in b"RP1 UART0 hello at 115200 8N1\r\n" {
        if b == b'\n' { uart_putc(b'\r'); }
        uart_putc(b);
    }
    loop {
        unsafe {
            if (reg_read(uart_reg(LSR)) & 1) != 0 {
                let c = reg_read(uart_reg(RBR_THR_DLL)) as u8;
                uart_putc(c);
            }
        }
    }
}
```

The UART register layout and GPIO function select come from the RP1 datasheet; on Pi 5 the Linux device tree shows these appearing at rp1/serial@30000 etc., confirming the BAR1+offset scheme for host access.[3] [1]

minicom wiring and settings

- Wire a 3.3V USB-to-UART adapter to the 40-pin header: GND to pin 6, adapter RX to pin 8 (GPIO14 TXD), adapter TX to pin 10 (GPIO15 RX); do not connect 5V or 3V3 between boards if the adapter is USB-powered.[2]
- On the host PC: minicom -b 115200 -D /dev/ttyUSB0 and select 115200 8N1, no flow control; if output is garbled, slightly adjust the divisor or confirm clk_uart is 48 MHz per the board's configuration.[2] [1]

Practical notes and resources

- Early boot consoles on Pi 5 often use the dedicated JST debug header between the HDMI ports, which is not the RP1 GPIO UART; for a bare-metal GPIO console over RP1, PCIe and RP1 must be initialized or preserved by prior firmware, as discussed by developers working with early console routing on Pi 5.[5] [6]
- RP1 peripheral bases, the GPIO function table, clock notes for clk_uart, and atomic APB access windows are documented in the RP1 Peripherals datasheet (pay attention to the BAR1 mapping and AP offsets), and kernel work notes that the AP reaches RP1 MMIO via BAR1 mapping of the endpoint.[4] [1]
- Linux device tree dumps for Pi 5 are a good sanity check for offsets and available UART instances on the 40-pin header, showing serial@30000, @34000, … under rp1 and pin groups like rp1_uart0_14_15 for TX/RX on GPIO14/15.[3]
- Some users have reported UART timing peculiarities at low baud rates on Pi 5; sticking to 115200 8N1 generally avoids issues, but test with the target peripheral if unusual baud rates are needed.[7]

References for implementation

- RP1 Peripherals datasheet: peripheral address map, UART bases, clk_uart ≈ 48 MHz, IO_BANK0 function select, and AP vs BAR1 mapping semantics.[1]
- Linux DT for Raspberry Pi 5: rp1 node, serial@30000..@40000, gpio@d0000 mapping that matches the BAR1+offset model used above.[3]
- Kernel RP1 notes: RP1 is a PCIe multifunction endpoint whose sub-devices (including UART) are reached via BAR1 MMIO from the AP.[4]
- UART wiring and minicom usage guide showing GND/TX/RX on pins 6/8/10 with 3.3V signalling.[2]
- Debug header orientation on Pi 5 (if comparing with early console paths that are not via RP1).[6]

⁂

How to boot custom bare metal os on raspbery pi 5 do thorough research

Boot a custom bare-metal OS on Raspberry Pi 5 by placing a 64-bit AArch64 binary named kernel_2712.img (or kernel8.img) and a config.txt on a FAT32 boot partition; the EEPROM firmware will load it directly and start execution without Linux or U-Boot required. For early serial output and debugging, use the dedicated 3-pin debug UART at 115200 8N1 or enable the RP1 UART on the 40-pin header via config.txt during development. [21] [22] [23]

Boot flow overview

- Raspberry Pi 5 boots from an on-board EEPROM that contains the VideoCore firmware; a config.txt must be present on the boot partition to mark it bootable, after which the firmware loads the kernel image and optional DTB. [21]
- On Pi 5 the default kernel filename is kernel_2712.img (optimized for BCM2712); if that file is absent the firmware falls back to the common 64-bit kernel name kernel8.img, and only 64-bit kernels are supported on Pi 5. [21]
- The firmware preloads an AArch64 payload as BL33 (kernel_2712.img or kernel8.img) and can be combined with Trusted Firmware-A BL31 via the armstub mechanism; boot messages by default go to the dedicated 3-pin debug UART at 115200 8N1. [23]

Prepare the boot media

- Create a single FAT32 boot partition and place config.txt plus the bare-metal image named kernel_2712.img (preferred) or kernel8.img at the root of the partition; this naming ensures the Pi 5 firmware recognizes and loads the image. [21]
- Pi 5 requires a config.txt on the boot partition; at minimum add a [pi5] section with kernel= if using a non-default name, and optionally set os_check=0 to bypass DTB compatibility checks for unconventional bare-metal payloads. [21]
- If using a device tree in the bare-metal OS, copy the appropriate bcm2712-rpi-5-b.dtb to the boot partition and optionally control load addresses with device_tree_address= and kernel_address= in config.txt, which the firmware honors on Pi 5. [24] [23]

Recommended config.txt for bare metal (Pi 5)

- Use a minimal file like:
  - [pi5] section
  - kernel=kernel_2712.img or rely on the default name kernel_2712.img if present. [21]
  - os_check=0 during bring-up if the payload is not shipping a standard DTB or deviates from expected model checks. [21]
  - enable_rp1_uart=1 to have firmware initialize RP1 UART0 at 115200 and skip resetting RP1, easing early serial on GPIO14/15; set pciex4_reset=0 to inherit PCIe/RP1 state if needed. [21]

Build the bare-metal image

- Cross-compile for AArch64 with aarch64-elf or aarch64-linux-gnu toolchains, producing a flat, uncompressed image named kernel_2712.img (or kernel8.img) to be placed on the boot partition; Pi 5 only boots 64-bit payloads.[21]
- The Pi 5 VPU preloads the AArch64 payload and selects load/entry addresses internally; if fixed addresses are required for a custom memory map, specify kernel_address= and device_tree_address= in config.txt to match the linker script.[23]
- For a robust EL2→EL1 transition and PSCI services, optionally combine the payload with TF-A: place bl31.bin as armstub (armstub=bl31.bin or rename to armstub8-2712.bin) so BL31 runs first, then hands execution to the preloaded BL33 image (kernel_2712.img or kernel8.img).[23]

Serial output and debugging

- The dedicated 3-pin JST-SH debug connector between the HDMI ports carries UART at 115200 8N1 by default during boot stages, making it the simplest way to observe early messages on Pi 5; connect RX, GND, TX as per the spec, and use a level-safe adapter supporting 1.8–3.3 V IO.[22] [23]
- For GPIO-header serial during bring-up, add enable_rp1_uart=1 so firmware initializes RP1 UART0 on GPIO14/15 at 115200 and preserves RP1 state; this avoids having to bring up PCIe/RP1 in the very first instructions of the bare-metal OS.[21]
- On the host side, attach a UART adapter and open a terminal at 115200 8N1; the firmware and TF-A both default to this configuration, allowing immediate console output before the OS initializes its own UART driver.[23]

Using a device tree (optional)

- If the bare-metal OS consumes a device tree, include bcm2712-rpi-5-b.dtb on the boot partition and allow the firmware to pass it to the payload, or set device_tree_address= to a fixed location if the OS expects it at a specific address.[23]
- The bcm2712-rpi-5-b.dtb is the model DTB used for Raspberry Pi 5; when customizing or experimenting, DTB changes can be derived from that baseline and deployed in the boot partition for the firmware to load.[24]

Supported boot media and variations

- Pi 5 firmware loads from supported media like SD and USB; a config.txt presence is required on the boot partition for the bootloader to consider the partition bootable on Pi 5.[23] [21]
- Advanced flows can chain TF-A as armstub to standardize EL setup and PSCI while still using the VPU to preload the actual bare-metal payload as BL33 from kernel_2712.img or kernel8.img.[23]

Troubleshooting

- If the board does not appear to boot, confirm the 64-bit image name (kernel_2712.img recommended) and that config.txt is present and readable on the FAT32 partition; Pi 5 only runs 64-bit kernels and requires config.txt to mark the partition bootable.[21]
- If device-tree compatibility checks block boot during experimentation, add os_check=0 under a [pi5] section, and restore to default once the DTB/payload matches expectations.[21]
- If serial output is missing, test the dedicated 3-pin debug UART at 115200 8N1, then, if using GPIO serial in early boot, enable RP1 UART via enable_rp1_uart=1 to get firmware-initialized UART on pins 14/15.[22] [23] [21]

Authoritative resources

- config.txt (Pi 5 specifics: kernel_2712.img default, 64-bit only, EEPROM-contained firmware, os_check, enable_rp1_uart, pciex4_reset, and required config.txt presence on Pi 5).[21]
- Trusted Firmware-A for Pi 5 (armstub/BL31 usage, kernel_2712.img/kernel8.img preload by VPU, device_tree_address/kernel_address controls, dedicated UART defaults).[23]
- 3-pin debug connector specification (connector type, pinout, and voltage guidance for the dedicated UART/SWD header used by Pi 5).[22]
- General bare-metal practices and image placement on Raspberry Pi (for context when producing kernel8.img-style payloads).[25]

⁂

Write minimal fully working setup in rust that would create a kernel.img to plaace in sd card. Only uart

Below is a minimal, fully working Rust bare-metal setup that builds a raw AArch64 kernel image which prints over the Raspberry Pi 5’s dedicated 3-pin debug UART at 115200 8N1; place the produced kernel.img on the SD card and set config.txt to load it on Pi 5.[50] [51]

What this uses

- The dedicated 3-pin debug UART that all boot stages use at 115200 8N1, so no RP1/PCIe bring-up is required for early printf-style output on Pi 5.[50]
- A direct MMIO write to the PL011-compatible debug UART at the SoC address shown by the Pi 5 device tree (serial@107d001000), which works as a minimal transmitter when the firmware already set the clock/baud in early boot.[52]

Project layout

- Files to create: Cargo.toml, .cargo/config.toml, build.rs, linker.ld, start.S, src/main.rs; this builds an ELF which is then objcopied into a flat kernel.img for Pi firmware to load via config.txt.[51]
- The firmware on Pi 5 loads kernel_2712.img by default, but any filename can be used by specifying kernel= in config.txt; here the output is kernel.img for clarity.[51]

linker.ld (link to 0x80000 and export __stack_top)

```
ENTRY(_start)

/* Link at 0x80000; firmware can be told to load here via kernel_address=0x80000 */
SECTIONS
{
  . = 0x80000;

  .text : {
    KEEP(*(.text._start))
    *(.text*)
  }

  .rodata : { *(.rodata*) }
  .data :   { *(.data*) }

  .bss (NOLOAD) : {
    __bss_start = .;
    *(.bss*)
    *(COMMON)
    __bss_end = .;
  }

  /* Simple 64 KiB stack after BSS */
  . = ALIGN(16);
  __stack_bottom = .;
  . += 64K;
  __stack_top = .;

  /DISCARD/ : { *(.comment*) *(.note*) }
}
```

Linking at 0x80000 keeps things simple, and Pi 5 firmware can be instructed to place the payload at this address via kernel_address= in config.txt if desired.[50] [51]

build.rs (assemble the startup stub)

```
fn main() {
    cc::Build::new()
        .file("start.S")
        .flag("-march=armv8-a")
```

```
        .compile("start");
}
```

A tiny assembly entry sets SP and calls into Rust; cargo builds the AArch64 ELF and the stub becomes the entry point symbol _start.[51]

start.S (set stack, call rust_main)

```
    .section .text._start
    .globl _start
    .align  4
_start:
    ldr     x0, =__stack_top
    mov     sp, x0

    bl      rust_main

1:  wfe
    b       1b
```

This ensures a valid stack before jumping into Rust, which is required since bare-metal entry provides no C runtime or stack by default.[51]

src/main.rs (no_std Rust, minimal PL011 TX)

```
#![no_std]
#![no_main]

use core::ptr::{read_volatile, write_volatile};

#[no_mangle]
pub extern "C" fn rust_main() -> ! {
    uart_puts("Hello from Rust on Raspberry Pi 5 debug UART!\r\n");
    loop {
        // simple idle loop
        unsafe { core::arch::asm!("wfe"); }
    }
}

// PL011-compatible debug UART (from DT: serial@107d001000)
const UART_BASE: usize = 0x107D_0010_00;

// Offsets (PL011)
const DR:   usize = 0x00; // Data register
const FR:   usize = 0x18; // Flag register
// FR bits: TXFF=1<<5 (TX FIFO full)

#[inline(always)]
fn mmio32(off: usize) -> *mut u32 {
    (UART_BASE + off) as *mut u32
}
```

```
fn uart_putc(c: u8) {
    unsafe {
        // Wait until TX FIFO not full
        while (read_volatile(mmio32(FR)) & (1 << 5)) != 0 {}
        write_volatile(mmio32(DR), c as u32);
    }
}

fn uart_puts(s: &str) {
    for b in s.bytes() {
        if b == b'\n' {
            uart_putc(b'\r');
        }
        uart_putc(b);
    }
}

#[panic_handler]
fn panic(info: &core::panic::PanicInfo) -> ! {
    uart_puts("panic: ");
    // very small panic print
    let _ = info;
    uart_puts("\r\n");
    loop {
        unsafe { core::arch::asm!("wfe"); }
    }
}
```

The firmware already initializes the dedicated debug UART at 115200 8N1, so a minimal TX routine can poll FR and write DR without reconfiguring clocks or divisors for the first “hello”.[52] [50]

Cargo.toml (no_std binary)

```
[package]
name = "pi5-bare-uart"
version = "0.1.0"
edition = "2021"

[dependencies]

[build-dependencies]
cc = "1.0"

[profile.release]
lto = true
codegen-units = 1
strip = "symbols"
```

This sets up a no_std binary with a build script to compile the assembly entry, producing an ELF that will be objcopied into a flat kernel image.[51]

.cargo/config.toml (AArch64 target and linker args)

```
[build]
target = "aarch64-unknown-none"

[target.aarch64-unknown-none]
rustflags = [
  "-Clink-arg=-Tlinker.ld",
  "-Crelocation-model=static",
  "-Cpanic=abort",
]
linker = "rust-lld"
```

Using the built-in aarch64-unknown-none target with rust-lld and a custom linker script yields a bare image suitable for Pi 5 firmware loading as kernel payload.[51]

Build and produce kernel.img

- Build a release ELF: cargo build --release --target aarch64-unknown-none, then convert to a raw image: llvm-objcopy -O binary target/aarch64-unknown-none/release/pi5-bare-uart kernel.img.[51]
- The Pi 5 firmware accepts a raw AArch64 payload as kernel_2712.img or kernel8.img by default, but kernel=kernel.img in config.txt works just as well on Pi 5.[50] [51]

SD card boot files (config.txt)

```
[pi5]
kernel=kernel.img
kernel_address=0x80000
```

Pi 5 requires config.txt on the boot partition and will then load the specified raw AArch64 binary; kernel_address= lets the linker and loader agree on the image's runtime address if a fixed link address is used.[50] [51]

Connecting the serial console

- Use the dedicated 3-pin JST-SH debug connector between the HDMI ports; it is the official early-boot console path on Pi 5 and runs at 115200 8N1 by default across all boot stages.[53] [50]
- Follow the debug connector spec for pinout and voltage levels; use a UART adapter that supports the correct logic level (1.8–3.3 V) and connect to a terminal at 115200 8N1 to see the greeting from rust_main.[53] [50]

Why this works on Pi 5

- The TF-A Pi 5 platform notes that the VPU preloads a 64-bit BL33 image named kernel_2712.img or kernel8.img, configurable via kernel=, and states all boot stages print to the dedicated UART debug port at 115200 8N1 by default, which this minimal payload leverages without extra hardware init.[50]
- The device tree for Pi 5 shows a PL011-compatible UART node at serial@107d001000, allowing simple polling writes to DR for output, which is sufficient for a “hello” and panic printing before adding a full driver.[52]

References

- TF-A Raspberry Pi 5 platform docs: preload names, kernel_address/device_tree_address, and default debug UART 115200 8N1.[50]
- Raspberry Pi config.txt docs: kernel= override, required presence on Pi 5, and general bootloader behavior for raw AArch64 images.[51]
- Raspberry Pi 3-pin debug connector specification: connector standardization, usage as UART or SWD, and safe electrical interface guidance.[53]
- Example DT excerpt for the BCM2712 debug UART base address (serial@107d001000) used here for minimal PL011 TX.[52]

⁂

Now what command should i run next to compile the code

Run these commands to build the AArch64 bare-metal ELF, convert it into a flat kernel.img, and place kernel.img plus config.txt on the SD card’s FAT32 boot partition for Raspberry Pi 5 to boot.[71] [72] [73]

Commands

1. Add the bare-metal AArch64 target for Rust: `rustup target add aarch64-unknown-none`.[74] [71]
2. Build the release binary for that target: `cargo build --release --target aarch64-unknown-none`.[74]
3. Convert the ELF to a raw image the Pi firmware loads: `llvm-objcopy -O binary target/aarch64-unknown-none/release/pi5-bare-uart kernel.img`.[72] [75]
4. Ensure the boot partition has a config.txt that selects the image, then copy files onto the SD card’s FAT32 partition:
   - Example config.txt entry: `kernel=kernel.img` (or use the Pi 5 default kernel_2712.img name).[73]
   - Copy files: `cp kernel.img /path/to/SD/boot/ && cp config.txt /path/to/SD/boot/`.[73]

5. Safely eject the SD, insert into Raspberry Pi 5, and power on to boot the bare-metal kernel.img specified in config.txt.[73]

⁂

warning: pi5-bare-uart@0.1.0: cc1: fatal error: start.S: No such file or directory

warning: pi5-bare-uart@0.1.0: compilation terminated.
error: failed to run custom build command for `pi5-bare-uart v0.1.0 (/mnt/Acer/Users/Haarit/Desktop/GitHubRepos/BuildingOS/uart)`
Caused by:
process didn't exit successfully: `/mnt/Acer/Users/Haarit/Desktop/GitHubRepos/BuildingOS/uart/target/release/build/pi5-bare-uart-c0e509f0ee174357/build-script-build` (exit status: 1)
--- stdout
OUT_DIR = Some(/mnt/Acer/Users/Haarit/Desktop/GitHubRepos/BuildingOS/uart/target/aarch64-unknown-none/release/build/pi5-bare-uart-09ffcf714912c0dc/out)
OPT_LEVEL = Some(3)
TARGET = Some(aarch64-unknown-none)
CARGO_ENCODED_RUSTFLAGS = Some()
HOST = Some(x86_64-unknown-linux-gnu)
cargo:rerun-if-env-changed=CC_aarch64-unknown-none
CC_aarch64-unknown-none = None
cargo:rerun-if-env-changed=CC_aarch64_unknown_none
CC_aarch64_unknown_none = None
cargo:rerun-if-env-changed=TARGET_CC
TARGET_CC = None
cargo:rerun-if-env-changed=CC
CC = None
cargo:rerun-if-env-changed=CROSS_COMPILE
CROSS_COMPILE = None
RUSTC_LINKER = None
cargo:rerun-if-env-changed=CC_ENABLE_DEBUG_OUTPUT
RUSTC_WRAPPER = None
cargo:rerun-if-env-changed=CRATE_CC_NO_DEFAULTS
CRATE_CC_NO_DEFAULTS = None
DEBUG = Some(false)
CARGO_CFG_TARGET_FEATURE = Some(neon)
cargo:rerun-if-env-changed=CFLAGS
CFLAGS = None
cargo:rerun-if-env-changed=TARGET_CFLAGS
TARGET_CFLAGS = None
cargo:rerun-if-env-changed=CFLAGS_aarch64_unknown_none
CFLAGS_aarch64_unknown_none = None

cargo:rerun-if-env-changed=CFLAGS_aarch64-unknown-none
CFLAGS_aarch64-unknown-none = None
cargo:warning=cc1: fatal error: start.S: No such file or directory
cargo:warning=compilation terminated.
--- stderr
error occurred in cc-rs: command did not execute successfully (status code exit status: 1): LC_ALL="C" "cc" "-O3" "-ffunction-sections" "-fdata-sections" "-Wall" "-Wextra" "-march=armv8-a" "-o" "/mnt/Acer/Users/Haarit/Desktop/GitHubRepos/BuildingOS/uart/target/aarch64-unknown-none/release/build/pi5-bare-uart-09ffcf714912c0dc/out/25ce5a424b770e84-start.o" "-c" "start.S"

The build failed because the build script asks the cc crate to compile start.S, but that file is missing; create start.S (or point build.rs at the correct path), then rebuild and objcopy the ELF into kernel.img for the SD card.[92] [93] [94] [95]

Quick fix: create start.S

- Create start.S in the project root alongside build.rs with the minimal entry code used earlier, then rebuild.[93] [92]
- Example command to create the file:

```
cat > start.S << 'EOF'
    .section .text._start
    .globl _start
    .align  4
_start:
    ldr     x0, =__stack_top
    mov     sp, x0
    bl      rust_main
1:  wfe
    b       1b
EOF
```

This satisfies cc::Build::new().file("start.S") in build.rs, which otherwise errors if the path does not exist.[92] [93]

Option: place it in src

- If keeping assembly in src/, move the file to src/start.S and change build.rs to .file("src/start.S") so the path resolves during the build script phase.[93] [92]
- Build scripts run before compiling Rust sources and must reference real files in the workspace, so adjusting the path in .file(...) is sufficient to fix the missing file error.[94] [93]

Rebuild and export image

- Rebuild for AArch64 bare metal:

```
rustup target add aarch64-unknown-none
cargo build --release --target aarch64-unknown-none
```

The aarch64-unknown-none target is the correct Rust target for a 64-bit bare-metal image on Pi 5.[96] [97]

- Convert the ELF to a flat image the Pi firmware can load:

```
llvm-objcopy -O binary target/aarch64-unknown-none/release/pi5-bare-uart kernel.img
```

Pi firmware accepts a raw AArch64 binary as the kernel image, hence the need for objcopy to produce kernel.img from the ELF.[95] [98]

Alternative without build.rs

- Remove the build.rs step and embed the entry assembly directly using global_asm!, avoiding an external start.S file.[99] [100]
- Example in src/main.rs:

```
#![no_std]
#![no_main]
use core::arch::global_asm;

global_asm!(r#"
    .section .text._start
    .globl _start
    .align  4
_start:
    ldr     x0, =__stack_top
    mov     sp, x0
    bl      rust_main
1:  wfe
    b       1b
"#);
```

The global_asm! macro emits module-level assembly so the linker sees _start without compiling a separate .S file.[100] [99]

Notes

- Ensure a C toolchain is installed, since the cc crate invokes a system compiler when compiling .S files in build.rs.[101] [92]
- If the build cache holds the previous failure, cleaning and rebuilding can help: cargo clean && cargo build --release --target aarch64-unknown-none.[97] [93]

⁂

1. https://datasheets.raspberrypi.com/rp1/rp1-peripherals.pdf
2. https://emteria.com/kb/connect-uart-rpi
3. https://gist.github.com/jens-maus/497e03cf1305ffe8a07e3196c27d6ebd
4. https://patchew.org/linux/cover.1724159867.git.andrea.porta@suse.com/5954e4dccc0e158cf434d2c281ad57120538409b.1724159867.git.andrea.porta@suse.com/
5. https://github.com/agherzan/meta-raspberrypi/issues/1434
6. https://docs.zephyrproject.org/latest/boards/raspberrypi/rpi_5/doc/index.html
7. https://piers.rocks/2023/11/18/raspberry-pi-5-uart-timings.html
8. https://www.jakelectronics.com/blog/learn-raspberry-pi-5-pinout-in-this-in-depth-comprehensive-guide
9. https://raspberry.piaustralia.com.au/blogs/news/exploring-the-raspberry-pi-rp1s-top-features-specs
10. https://picockpit.com/raspberry-pi/i-read-the-rp1-documentation-so-you-dont-have-to/
11. https://datasheets.raspberrypi.com
12. https://www.raspberrypi.com/documentation/computers/config_txt.html
13. https://www.raspberrypi.com/documentation/computers/configuration.html
14. https://www.cnx-software.com/2023/10/07/raspberry-pi-rp1-datasheet-block-diagram/
15. https://github.com/worproject/rpi5-uefi
16. https://www.youtube.com/watch?v=nBtOEmUqASQ
17. https://www.scribd.com/document/808148682/Rp1-Peripherals
18. https://community.element14.com/products/raspberry-pi/m/files/148385
19. https://www.raspberrypi.com/documentation/computers/raspberry-pi.html
20. https://www.raspberrypi.com/news/rp1-the-silicon-controlling-raspberry-pi-5-i-o-designed-here-at-raspberry-pi/
21. https://www.raspberrypi.com/documentation/computers/config_txt.html
22. https://datasheets.raspberrypi.com/debug/debug-connector-specification.pdf
23. https://trustedfirmware-a.readthedocs.io/en/v2.12/plat/rpi5.html
24. https://www.jeffgeerling.com/blog/2023/how-customize-dtb-device-tree-binary-on-raspberry-pi
25. http://wiki.osdev.org/Raspberry_Pi_Bare_Bones
26. https://www.youtube.com/watch?v=FnpkJj8v3ww
27. https://components.ubitap.com/raspi_kits?store-page=USB-UART-Cable-for-Raspberry-Pi-5-p630800274
28. https://github.com/raspberrypi/linux/issues/6504
29. https://metebalci.com/blog/bare-metal-rpi3-network-boot/
30. https://www.waveshare.com/wiki/Pi_UART_Debugger
31. https://retropie.org.uk/forum/topic/34627/raspberry-pi-5-official-announcement/281
32. https://www.rpi4os.com/part2-building/
33. https://pip.raspberrypi.com/documents/RP-008180-DS-cm5-datasheet.pdf
34. https://www.raspberrypi.com/documentation/computers/configuration.html
35. https://www.mouser.com/datasheet/2/635/cm5_datasheet-3537283.pdf

36. https://www.robotics.org.za/SC0889
37. https://forum.endeavouros.com/t/pi-5-generic-kernel-help/67973
38. https://www.reddit.com/r/raspberrypihelp/comments/1g51nck/devicetree_file_bcm2712rpi5bdtb_not_found_cannot/
39. https://github.com/raspberrypi/linux/issues/6520
40. https://emlogic.no/2024/09/linux-drivers-getting-started-with-gpio-on-raspberry-pi-5/
41. https://www.linkedin.com/pulse/raspberry-pi-5-kernel-build-guide-david-zhu-pkihc
42. https://docs.zephyrproject.org/latest/boards/raspberrypi/rpi_5/doc/index.html
43. https://www.raspberrypi.com/documentation/computers/linux_kernel.html
44. https://github.com/MichaIng/DietPi/issues/6676
45. https://www.linkedin.com/pulse/updating-cross-compiled-kernel-raspberry-pi-ssh-sd-card-david-zhu-efqgc
46. https://developer.arm.com/documentation/ka006096/latest/
47. https://s-matyukevich.github.io/raspberry-pi-os/docs/lesson01/rpi-os.html
48. https://www.raspberrypi.com/documentation/computers/raspberry-pi.html
49. https://github.com/raspiblitz/raspiblitz/issues/4346
50. https://trustedfirmware-a.readthedocs.io/en/latest/plat/rpi5.html
51. https://www.raspberrypi.com/documentation/computers/config_txt.html
52. https://www.linkedin.com/pulse/comprehensive-guide-early-boot-uart-configuration-tty-david-zhu-2qvec
53. https://datasheets.raspberrypi.com/debug/debug-connector-specification.pdf
54. https://piers.rocks/2023/11/18/raspberry-pi-5-uart-timings.html
55. http://www.circuitvalley.com/2024/09/raspberrypi5debuguartconnectorcablepartterminalpinoutpins.html
56. https://docs.zephyrproject.org/latest/boards/raspberrypi/rpi_5/doc/index.html
57. https://trustedfirmware-a.readthedocs.io/en/v2.9/plat/marvell/armada/uart-booting.html
58. https://www.waveshare.com/wiki/Pi_UART_Debugger
59. https://raspberrytips.com/enable-uart-on-raspberry-pi/
60. https://github.com/worproject/rpi5-uefi
61. https://www.youtube.com/watch?v=17a5ddSQOVE
62. https://docs.nordicsemi.com/bundle/ncs-2.9.0/page/zephyr/boards/raspberrypi/rpi_5/doc/index.html
63. https://openrt.gitbook.io/open-surfacert/surface-rt/hardware/uart/raspberry-pi-uart-setup
64. https://www.raspberrypi.com/documentation/microcontrollers/debug-probe.html
65. https://www.instructables.com/Connect-the-Raspberry-Pi-to-network-using-UART/
66. https://www.raspberrypi.com/documentation/computers/configuration.html
67. https://docs.nordicsemi.com/bundle/ncs-3.0.1/page/zephyr/boards/raspberrypi/rpi_5/doc/index.html
68. https://datasheets.raspberrypi.com/bcm2711/bcm2711-peripherals.pdf
69. https://www.reddit.com/r/raspberry_pi/comments/1ev61jg/datasheet_for_bcm2712/
70. https://www.electronicwings.com/raspberry-pi/raspberry-pi-uart-communication-using-python-and-c

71. https://doc.rust-lang.org/nightly/rustc/platform-support/aarch64-unknown-none.html
72. https://www.rpi4os.com/part2-building/
73. https://www.raspberrypi.com/documentation/computers/config_txt.html
74. https://krinkinmu.github.io/2020/12/13/adding-rust-to-aarch64.html
75. http://wiki.osdev.org/Raspberry_Pi_Bare_Bones
76. https://stackoverflow.com/questions/64893460/cross-compiling-kernel-for-raspberry-pi-4
77. https://lowenware.com/blog/aarch64-bare-metal-program-in-rust/
78. https://stackoverflow.com/questions/78845548/rust-cross-compile-aarch64-unknown-linux-gnu-linker-error
79. https://doc.rust-lang.org/beta/rustc/platform-support/x86_64-unknown-none.html
80. https://github.com/briansmith/ring/issues/1414
81. https://github.com/RPi-Distro/repo/issues/344
82. https://www.reddit.com/r/rust/comments/1j6kv8v/help_zero_2w_bare_metal_rust/
83. https://betterprogramming.pub/cross-compiling-rust-from-mac-to-linux-7fad5a454ab1
84. https://www.raspberrypi.com/documentation/computers/linux_kernel.html
85. https://www.reddit.com/r/rust/comments/zx1d20/any_clever_ideas_to_make_debugging_bare_metal/
86. https://doc.rust-lang.org/rustc/platform-support.html
87. https://www.reddit.com/r/rust/comments/1gj9the/crosscompile_aarch64unknownlinuxgnu_to/
88. https://stackoverflow.com/questions/70559555/rust-bare-metal-cross-compilation-for-aarch64-cant-find-crate-for-core
89. https://users.rust-lang.org/t/targeting-aarch64-unknown-none-elf/6351
90. https://jcomes.org/aarch64-os-hello_world
91. https://users.rust-lang.org/t/cross-compile-for-aarch64-unknown-linux-gnu-on-windows/79654
92. https://docs.rs/cc/latest/cc/struct.Build.html
93. https://doc.rust-lang.org/cargo/reference/build-scripts.html
94. https://doc.rust-lang.org/cargo/reference/build-script-examples.html
95. https://www.rpi4os.com/part2-building/
96. https://krinkinmu.github.io/2020/12/13/adding-rust-to-aarch64.html
97. https://doc.rust-lang.org/nightly/rustc/platform-support/aarch64-unknown-none.html
98. http://wiki.osdev.org/Raspberry_Pi_Bare_Bones
99. https://doc.rust-lang.org/beta/core/arch/macro.global_asm.html
100. https://doc.rust-lang.org/reference/inline-assembly.html
101. https://crates.io/crates/cc/1.0.62
102. https://docs.rs/cc
103. https://users.rust-lang.org/t/find-path-to-generated-libraries-by-cc/126494
104. https://rust-lang-nursery.github.io/rust-cookbook/development_tools/build_tools.html
105. https://lowenware.com/blog/aarch64-bare-metal-program-in-rust/
106. https://docs.rust-embedded.org/embedonomicon/asm.html
107. https://rust-lang.github.io/rfcs/1548-global-asm.html

108. https://stackoverflow.com/questions/70559555/rust-bare-metal-cross-compilation-for-aarch64-cant-find-crate-for-core
109. https://stackoverflow.com/questions/39219961/how-to-get-assembly-output-from-building-with-cargo
110. https://github.com/rust-lang/rust/issues/134758
111. https://users.rust-lang.org/t/targeting-aarch64-unknown-none-elf/6351
112. https://github.com/rust-lang/stacker/issues/80
113. https://doc.rust-lang.org/core/arch/aarch64/index.html